

动态规划

《算法设计与分析》

信息科学与技术学院 张德富



05



动态规划的基本性质



五、动态规划的基本性质

- 找最优子结构性质具有下列规律：
 - (1) 问题的划分依赖于一个决策，这个决策可以将原问题分解成一个或多个相似的子问题；
 - (2) 假定一个导致最优解的决策，先不关心这个最优决策是如何得到的；
 - (3) 对给定的这个最优决策，确定所划分的子问题以及如何方便地分析子问题解空间的变化。



最优子结构的参数

- 最优子结构主要涉及下面两个问题:

- (1) 当求解原问题时,需要用到多少个自问题?

- 装配线调度: 一个子问题(获得最短装配时间的装配线)
 - 矩阵链乘积: 两个子问题 ($A_{i..k}, A_{k+1..j}$)

- (2) 在确定哪些子问题的解包含在最优解中, 有多少种的选择

- 装配线调度: 两个选择(装配线1 或者装配线2)
 - 矩阵链乘积: 对于 k 有 $j - i$ 选择



最优子结构的参数

- 一个动态规划算法的运行时间取决于
 - (1) 所有子问题的数量;
 - (2) 要在多少个选择里面做最优的决策。

- 装配线调度

- $\Theta(n)$ 个子问题(n 个装配点)
 - 对每个子问题, 有2个选择

总共 $\Theta(n)$

- 矩阵链乘积:

- $\Theta(n^2)$ 个子问题($1 \leq i \leq j \leq n$)
 - 至多 $n-1$ 个选择

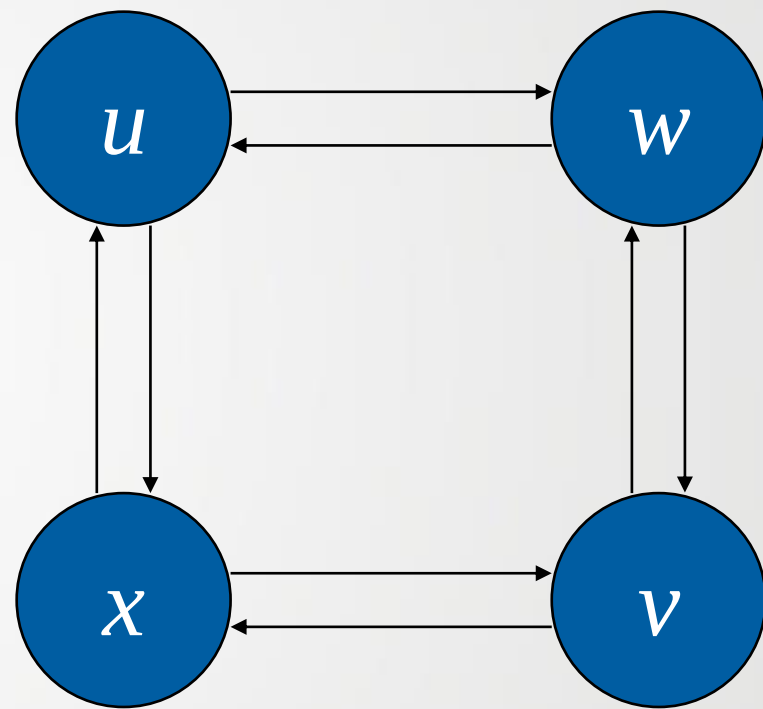
总共 $\Theta(n^3)$



细节

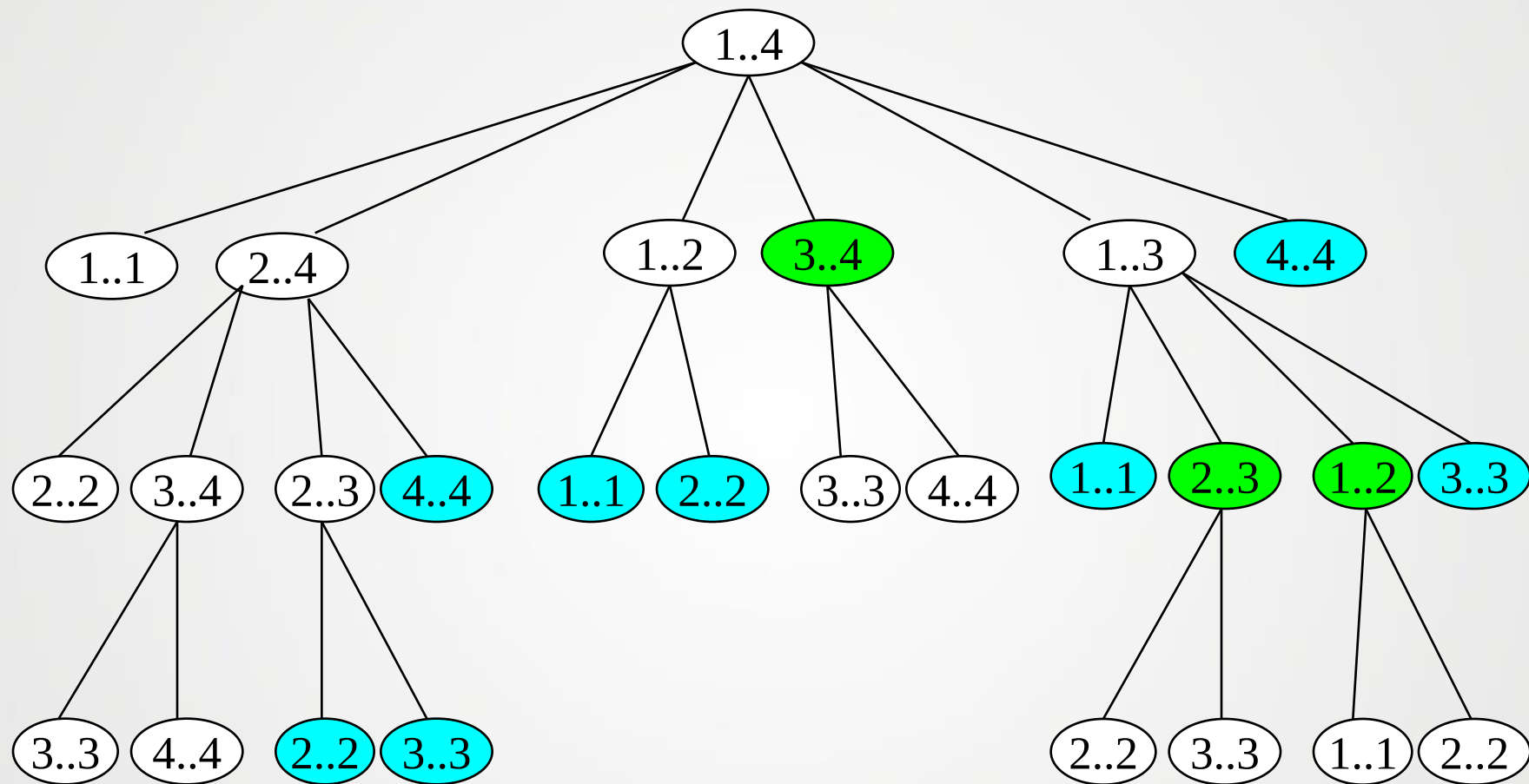
- 在使用动态规划算法来求解问题时，如果问题不具有最优子结构性质，而想当然的认为有，就会适得其反，达不到求解的效果。
- 给定一个有向图 $G = (V, E)$ 和顶点 $u, v \in V$ ，考虑下面两个问题：
 - ① 不带权的最短路径问题:寻找一条从 u 到 v 含有最少边数的路径。这样的路径必须是简单路径即序列中顶点不重复出现的路径，要不然从这个路径中删去一个环会产生一条含有更少边数的路径。
 - ② 不带权的最长简单路径:寻找一条从 u 到 v 含有最多边数的路径。这条路径必须是简单路径，要不然我们可以多次地绕着一个环遍历，从而得到一条含有任意多边数的路径。

- 对于不带权的最短路径问题，问题具有最优子结构性质。解一个子问题跟解另一个子问题是独立。
- 对于不带权的最长路径问题，不带权的最长路径问题则不具有最优子结构性质。解一个子问题跟解另一个子问题是不独立的。





重叠子问题





备忘录

- 其**基本思想**与递归的思想相似，采用自顶向下的策略，不同地是把第一次计算过的子问题的解保存在表中，以后每次遇到该子问题的时候只需查询之前表中填入的值即可。

LookUpChain(p, i, j)

```
1 if  $m[i, j] < \infty$  then return  $m[i, j]$ 
2 if  $i = j$  then
3      $m[i, j] \leftarrow 0$ 
4 else for  $k \leftarrow i$  to  $j - 1$  do
5      $q \leftarrow \text{LookUpChain}(p, i, k) + \text{LookUpChain}(p, k + 1, j) + p_{i-1}p_kp_j$ 
6     if  $q < m[i, j]$  then
7          $m[i, j] \leftarrow q$ 
8 return  $m[i, j]$ 
```

MemoizedMatrixChain(p)

```
1 for  $i \leftarrow 1$  to  $n$  do
2     for  $j \leftarrow i$  to  $n$  do
3          $m[i, j] \leftarrow \infty$ 
4 return  $\text{LookUpChain}(p, 1, n)$ 
```



动态规划 vs. 备忘录

动态规划算法：

- 采用自底向上的求解方式，不需要涉及函数的递归调用等代价，维护表的代价较小，用表的形式降低了时间和空间的要求。

备忘录方法：

- 采用自顶向下的求解方式，有些子问题并不需要求解，但是仍然涉及函数调用和参数的传递。



结论

- **分治方法:**

1. 子问题是相互独立的。
2. 子问题被重复计算。

- **相同点:**

问题都是被划分成一个或是多个子问题, 然后把子问题的解组合起来。

- **动态规划 (DP)**

1. 子问题是不独立的。
2. 子问题仅被执行一次。



结论

- 通过以下方法可以减少DP的计算：
 - 以自底向上的方法解决子问题。
 - 把第一次已得到解答的子问题的解保存起来。
 - 当再次遇到该子问题时，检索这个解。
- 关键：构造最优解结构。



作业

- 编程实现装配线调度问题

谢 谢 大 家